МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ
РОССИЙСКОЙ ФЕДЕРАЦИИ
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ БЮДЖЕТНОЕ
ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
«МОСКОВСКИЙ АВИАЦИОННЫЙ ИНСТИТУТ
(национальный исследовательский университет)»
Центр образовательных программ топ-уровня в сфере
информационных технологий Образовательного центра Института №8

Лабораторная работа №2 по курсу

«Операционные системы и архитектура компьютера»

«Управление процессами и обмен данными между процессами посредством каналов»

Студент: Цыганок Юлия Дмитриевна
Группа: М8О–205БВ–25
Преподаватель: Филимонов Н. С.
Оценка: _____
Дата: _____
Подпись: _____

Москва, 2026

1. ВВЕДЕНИЕ

1.1 Цель работы

Приобретение практических навыков в управлении процессами в операционной системе и обеспечении обмена данными между процессами посредством каналов.

1.2 Задание

Составить и отладить программу на языке С, осуществляющую работу с процессами и взаимодействие между ними. Программа родительский процесс должна создавать для решения задачи дочерние процессы. Взаимодействие между процессами осуществляется через каналы. Необходимо обрабатывать системные ошибки, которые могут возникнуть в результате работы. Вариант 20: правило фильтрации: строки длины больше 10 символов отправляются в pipe2, иначе в pipe1. Дочерние процессы инвертируют строки.

2. ХОД РАБОТЫ

2.1 Описание варианта и схемы взаимодействия

В соответствии с вариантом 20 (представлен на рис. 1) требуется реализовать программу, в которой:

- Родительский процесс запрашивает у пользователя имена двух файлов, создает два канала и запускает два дочерних процесса;
- Родитель читает строки от пользователя. Если длина строки > 10 символов — она пишется в pipe2, иначе — в pipe1.
- child1 читает строки из pipe1, инвертирует каждую строку и пишет результат в первый файл.
- child2 читает строки из pipe2, инвертирует каждую строку и пишет результат во второй файл.
- Родитель и дочерние процессы — разные программы, связь между ними обеспечивается через fork() + execve()

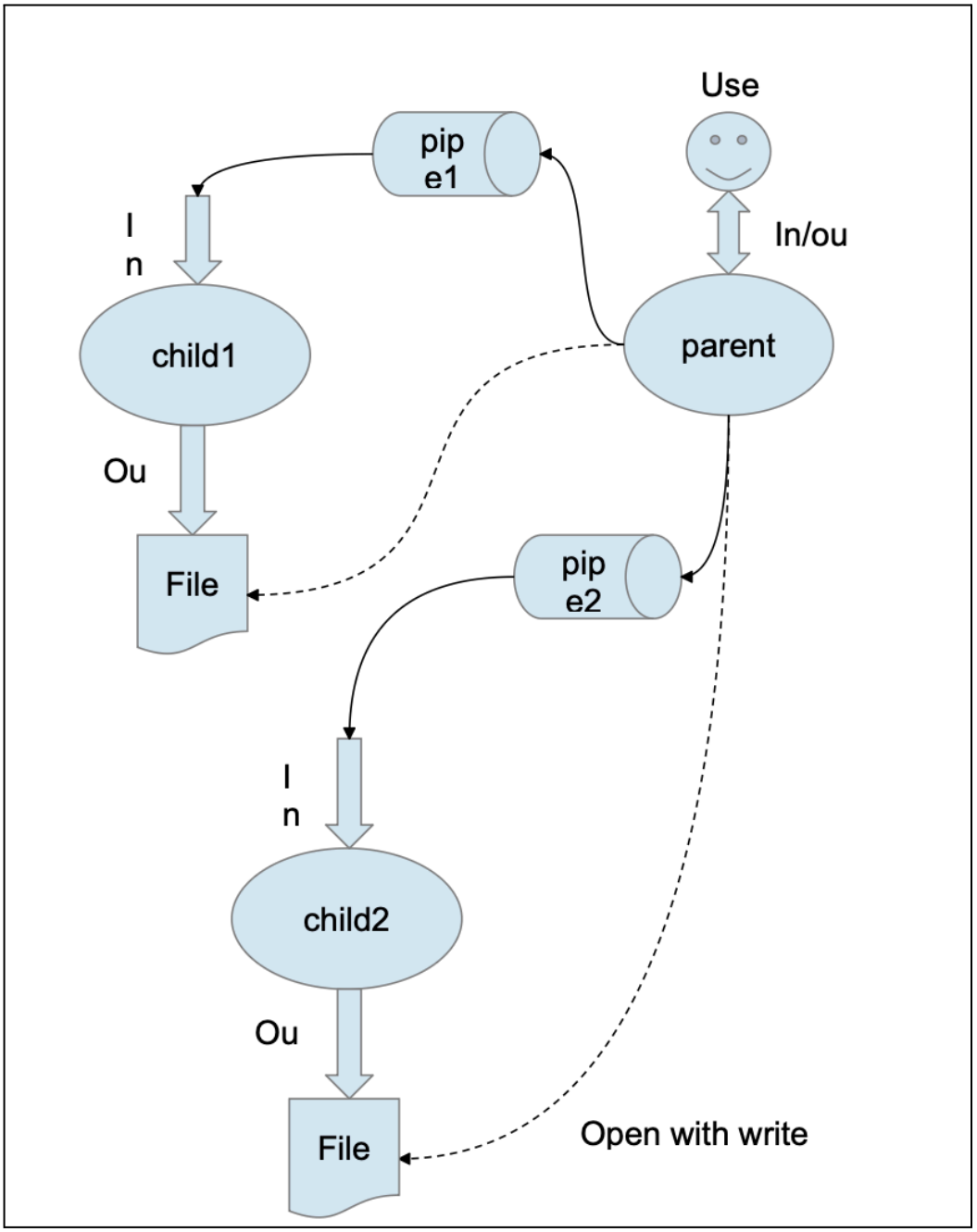


Рис. 1. Схема варианта 20, группа вариантов 5

2.2 Используемые системные вызовы

Для реализации поставленной задачи в программе используются стандартные системные вызовы операционной системы Linux, обеспечивающие создание процессов, организацию межпроцессного взаимодействия через каналы и работу с файлами.

pipe(int fd[2])	Создаёт однонаправленный канал. fd[0] — чтение, fd[1] — запись
fork()	Создаёт дочерний процесс (копию родителя)

execve(path, argv, envp)	Заменяет образ памяти процесса новой программой
dup2(oldfd, newfd)	Переназначает файловый дескриптор (перенаправление stdin)
waitpid(pid, &status, 0)	Ожидает завершения дочернего процесса
open(path, flags, mode)	Открывает/создаёт файл
read(fd, buf, n) / write(fd, buf, n)	Чтение/запись байтов
close(fd)	Закрывает файловый дескриптор
perror(str)	Выводит сообщение об ошибке

2.3 Методика выполнения работы

Для выполнения работы использовалась операционная система Linux - подсистема WSL 2 в Windows 11, дистрибутив Ubuntu. Компиляция выполнялась компилятором GCC версии 13.

Порядок выполнения работы:

1. Созданы три исходных файла на языке C: parent.c, child1.c, child2.c.
2. Компиляция трёх программ и запуск родительской программы (представлена на рис. 2)

```
2$ gcc -Wall -Wextra parent.c -o parent
2$ gcc -Wall -Wextra child1.c -o child1
2$ gcc -Wall -Wextra child2.c -o child2
2$ ./parent
```

Рис. 2. Компиляция и запуск

3. Ввод имён файлов для child1 и child2.
4. Ввод строк разной длины для проверки фильтрации.
5. Завершение ввода клавишами Ctrl+D.
6. Проверка результатов в файлах через cat.

2.4 Описание работы программы

Код программы и каждого процесса представлен в приложении отчёта по лабораторной работе №2.

Родительский процесс:

1. Запрашивает у пользователя имена двух файлов.
2. Создаёт два канала через pipe(pipe1) и pipe(pipe2).
3. Через fork() создает два дочерних процесса.
4. В каждом ребёнке:
    - закрывает ненужные концы каналов;
    - перенаправляет stdin на нужный канал через dup2(pipe1[0], STDIN_FILENO);
    - через execve() запускает ./child1 или ./child2, передавая имя файла как аргумент.
5. В родителе закрывает читающие концы каналов.
6. В цикле читает строки из stdin:
    - если длина строки больше 10 — пишет в pipe2;
    - иначе — в pipe1.
7. По окончании ввода закрывает пишущие концы каналов — дети получают EOF.
8. Через waitpid() ожидает завершения обоих детей.

Дочерние процессы (child1, child2)

1. Получают имя файла через argv[1].
2. Открывают файл на запись.
3. В цикле читают строки из stdin.
4. Каждую строку инвертируют.
5. Записывают результат в файл.
6. По получении EOF закрывают файл и завершаются.

В родителе и в каждом ребёнке закрываются все концы каналов, которые не используются. Это необходимо, чтобы читатель получил EOF, иначе программа зависнет на fgets/read.

ВЫВОД

В ходе выполнения лабораторной работы были приобретены практические навыки управления процессами в операционной системе Linux и организации обмена данными между процессами через каналы.

Была составлена и отлажена программа на языке Си, состоящая из трёх отдельных программ: родительского процесса parent и двух дочерних процессов child1, child2.

Родительский процесс создаёт два канала и два дочерних процесса через системные вызовы pipe() и fork(), после чего заменяет образ памяти каждого ребёнка программой child1/child2 через execve(). Стандартный ввод каждого ребёнка перенаправляется на соответствующий канал через dup2().

Родительский процесс читает строки, вводимые пользователем, и в зависимости от длины строки направляет их в первый или второй канал в соответствии с правилом фильтрации варианта 20 (порог — 10 символов). Дочерние процессы инвертируют полученные строки и записывают результаты в файлы, имена которых переданы им через аргументы командной строки.

В результате работы были изучены следующие системные вызовы: pipe, fork, execve, dup2, waitpid, open, read, write, close. Освоены принципы работы с файловыми дескрипторами и механизмы перенаправления стандартных потоков ввода-вывода. Отработаны приёмы обработки системных ошибок через perror(). Все поставленные задачи выполнены в полном объёме, программа работает корректно.

ПРИЛОЖЕНИЕ

Parent.c

```c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <fcntl.h>
#include <sys/wait.h>
#include <sys/types.h>
#include <errno.h>

#define MAX_LEN   4096
#define THRESHOLD 10

int main(void)
{
    char file1[256], file2[256];
    printf("Имя файла для child1: ");
    fflush(stdout);
    if (fgets(file1, sizeof(file1), stdin) == NULL) {
        fprintf(stderr, "Ошибка чтения\n");
        return 1;
    }
    file1[strcspn(file1, "\n")] = '\0';

    printf("Имя файла для child2: ");
    fflush(stdout);
    if (fgets(file2, sizeof(file2), stdin) == NULL) {
        fprintf(stderr, "Ошибка чтения\n");
        return 1;
```

```
    }
    file2[strcspn(file2, "\n")] = '\0';



    int pipe1[2], pipe2[2];
    if (pipe(pipe1) < 0) {
        perror("pipe1");
        return 1;
    }
    if (pipe(pipe2) < 0) {
        perror("pipe2");
        return 1;
    }
    pid_t pid1 = fork();
    if (pid1 < 0) {
        perror("fork child1");
        return 1;
    }


    if (pid1 == 0) {
        close(pipe1[1]);
        close(pipe2[0]);
        close(pipe2[1]);
        if (dup2(pipe1[0], STDIN_FILENO) < 0) {
            perror("dup2 child1");
            _exit(1);
        }
        close(pipe1[0]);
        char *argv[] = {"./child1", file1, NULL};
        char *envp[] = {NULL};
```

```
        execve("./child1", argv, envp);
        perror("execve child1");
        _exit(1);
    }
    pid_t pid2 = fork();
    if (pid2 < 0) {
        perror("fork child2");
        return 1;
    }

    if (pid2 == 0) {
        close(pipe2[1]);
        close(pipe1[0]);
        close(pipe1[1]);

        if (dup2(pipe2[0], STDIN_FILENO) < 0) {
            perror("dup2 child2");
            _exit(1);
        }
        close(pipe2[0]);

        char *argv[] = {"./child2", file2, NULL};
        char *envp[] = {NULL};
        execve("./child2", argv, envp);

        perror("execve child2");
        _exit(1);
    }
    close(pipe1[0]);
    close(pipe2[0]);
```

```
    printf("\nСтроки:\n");


    char buf[MAX_LEN];
    while (fgets(buf, sizeof(buf), stdin) != NULL) {
        size_t len = strlen(buf);
        int has_nl = (len > 0 && buf[len - 1] == '\n');
        if (has_nl) {
            buf[len - 1] = '\0';
            len--;
        }


        int target_fd;
        if ((int)len > THRESHOLD) {
            target_fd = pipe2[1];
        } else {
            target_fd = pipe1[1];
        }
        if (write(target_fd, buf, len) < 0) {
            perror("write body");
            break;
        }
        if (write(target_fd, "\n", 1) < 0) {
            perror("write nl");
            break;
        }
    }
    close(pipe1[1]);
    close(pipe2[1]);
    int status;
    waitpid(pid1, &status, 0);
```

```
    waitpid(pid2, &status, 0);


    printf("\nЗапсиано %s, %s\n", file1, file2);
    return 0;
}
```

Child1.c

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <fcntl.h>
#include <errno.h>

#define MAX_LEN 4096

static void reverse_string(char *s)
{
    int n = (int)strlen(s);
    for (int i = 0; i < n / 2; i++) {
        char t = s[i];
        s[i] = s[n - 1 - i];
        s[n - 1 - i] = t;
    }
}

int main(int argc, char *argv[])
{
    if (argc != 2) {
        return 1;
```

```
    }

    int out_fd = open(argv[1], O_WRONLY | O_CREAT | O_TRUNC, 0644);
    if (out_fd < 0) {
        perror("open");
        return 1;
    }

    char buf[MAX_LEN];
    while (fgets(buf, sizeof(buf), stdin) != NULL) {
        size_t len = strlen(buf);

        int has_nl = (len > 0 && buf[len - 1] == '\n');
        if (has_nl) {
            buf[len - 1] = '\0';
        }

        reverse_string(buf);

        if (write(out_fd, buf, strlen(buf)) < 0) {
            perror("write");
            break;
        }
        if (has_nl) {
            if (write(out_fd, "\n", 1) < 0) {
                perror("write nl");
                break;
            }
        }
    }
```

```
    close(out_fd);
    return 0;
}
```

Child2.c

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <fcntl.h>
#include <errno.h>

#define MAX_LEN 4096

static void reverse_string(char *s)
{
    int n = (int)strlen(s);
    for (int i = 0; i < n / 2; i++) {
        char t = s[i];
        s[i] = s[n - 1 - i];
        s[n - 1 - i] = t;
    }
}

int main(int argc, char *argv[])
{
    if (argc != 2) {
        return 1;
    }
```

```c
int out_fd = open(argv[1], O_WRONLY | O_CREAT | O_TRUNC, 0644);
if (out_fd < 0) {
    perror("open");
    return 1;
}

char buf[MAX_LEN];
while (fgets(buf, sizeof(buf), stdin) != NULL) {
    size_t len = strlen(buf);

    int has_nl = (len > 0 && buf[len - 1] == '\n');
    if (has_nl) {
        buf[len - 1] = '\0';
    }

    reverse_string(buf);

    if (write(out_fd, buf, strlen(buf)) < 0) {
        perror("write");
        break;
    }
    if (has_nl) {
        if (write(out_fd, "\n", 1) < 0) {
            perror("write nl");
            break;
        }
    }
}
```

```
    close(out_fd);

    return 0;

}
```